

Feedback

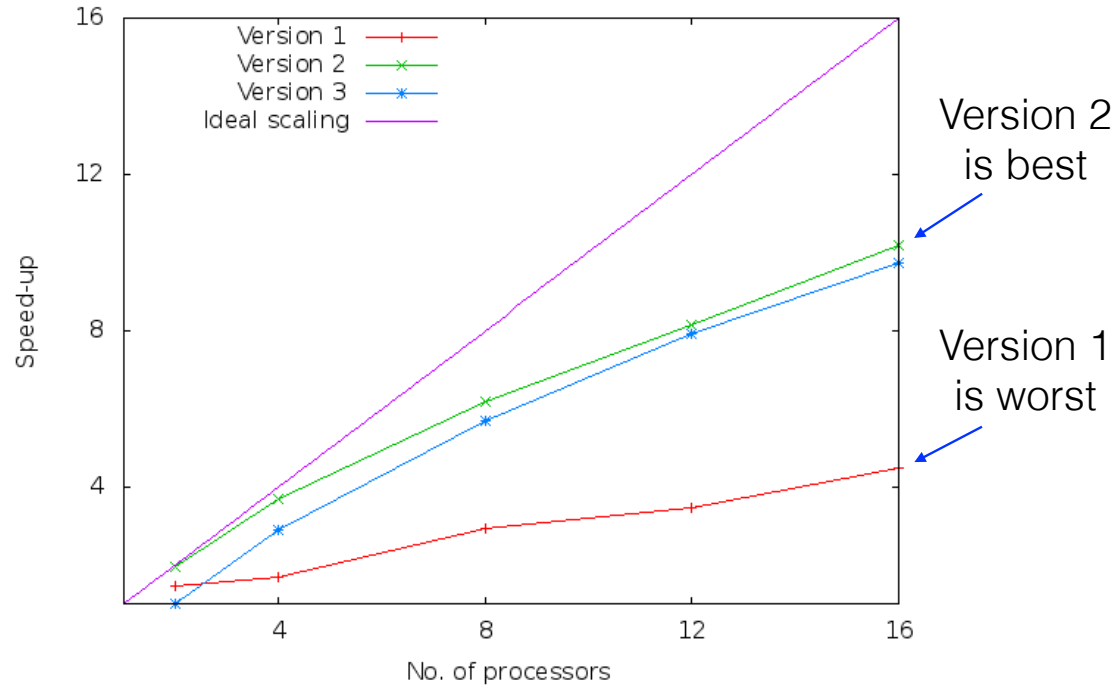
Workshop 8

Workshop 8 feedback

- In the eighth workshop we looked at the parallel performance of MPI programs which calculate the Mandelbrot set
- There were three versions of the program which distribute work in different ways
- The scaling was tested on 1 node (2,4,8,12,16 processors) and 2-nodes (32 processors)

Workshop 8

Single node scaling results:



Workshop 8

Question 2: Which version of the program performs worst and how does it distribute the workload?

```
if (verbose) {printf("Process %d of %d starting calculation\n", myRank, nProcs);}

/* Initialise to nIter zero as we will use a reduce to collate results into this array*/
for (i=0; i<nRe+1; i++){
    for (j=0; j<nIm+1; j++){
        nIter[i][j]=0;
    }
}

/* Divide work between MPI processes by splitting the i loop into nProcs pieces */
int nRePerProc = nRe / nProcs;
int imin = nRePerProc * myRank;
int imax = nRePerProc * (myRank+1) - 1;
if ( myRank==nProcs-1 ) imax=nRe;

if (verbose) {printf("Process %d will calculate %d to %d\n", myRank, imin, imax);}

/* Loop over real and imaginary axes */
for (i=imin; i<imax+1; i++){
    for (j=0; j<nIm+1; j++){
        z_Re = ( (float) i) / ( (float) nRe) * (z_Re_max - z_Re_min) + z_Re_min;
        z_Im = ( (float) j) / ( (float) nIm) * (z_Im_max - z_Im_min) + z_Im_min;
        z0 = z_Re + z_Im*I;
        z = z0;

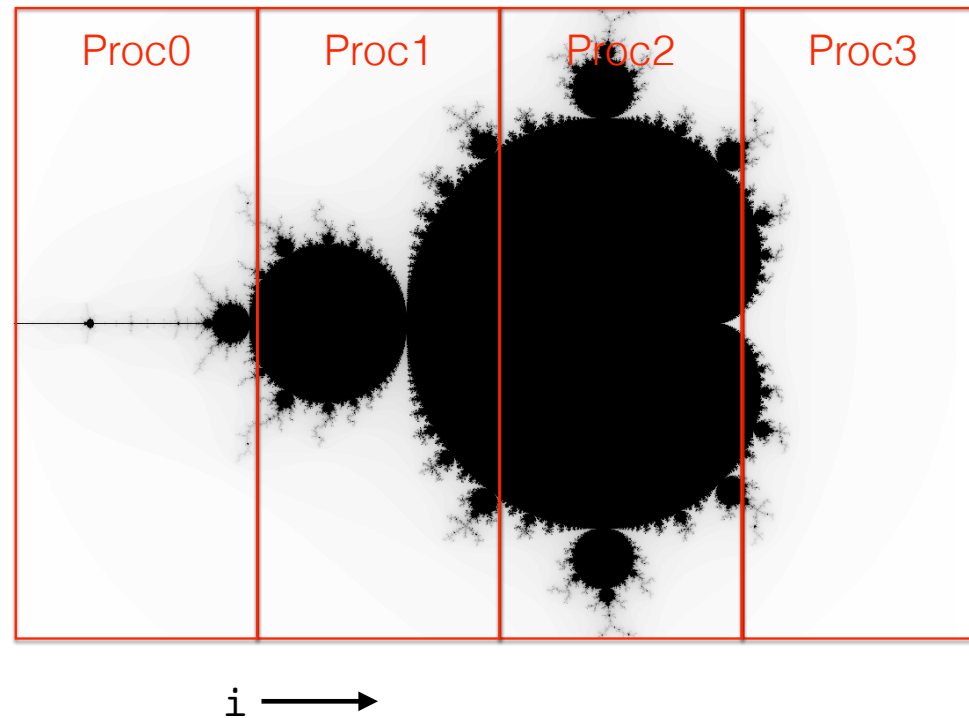
        /* Iterate up to a maximum number or bail out if mod(z) > 2 */
        k=0;
        while(k<maxIter){
            nIter[i][j] = k;
            if (cabs(z) > 2.0)
                break;
            z = z*z + z0;
            k++;
        }
    }
}
```

Each process works out which iterations to do based on its rank

In case iterations do not divide exactly by number of processes

Workshop 8

For example with 4 MPI processes:



Workshop 8

An OpenMP version

```
printf("Starting calculation\n");

/* Loop over real and imaginary axes */
#pragma omp parallel for default(none) private(i, j, z_Re, z_Im, z0, z, k, thisIter) shared(nIter) \
schedule(static)
for (i=0; i<nRe+1; i++){
    for (j=0; j<nIm+1; j++){
        z_Re = ( (float) i) / ( (float) nRe) * (z_Re_max - z_Re_min) + z_Re_min;
        z_Im = ( (float) j) / ( (float) nIm) * (z_Im_max - z_Im_min) + z_Im_min;
        z0 = z_Re + z_Im*I;
        z = z0;

        /* Iterate up to a maximum number or bail out if mod(z) > 2 */
        k=0;
        while(k<maxIter){
            thisIter = k;
            z = z*z + z0;
            nIter[i][j] = thisIter;
            if (cabs(z) > 2.0)
                break;
            k++;
        }
    }
}
```

Distribute outer loop to OpenMP threads
in equal size chunks

Load balance

- We expect this decomposition of the work to be poorly load balanced - version 1 is like OpenMP with a static loop schedule
- We can use a profiling tool to measure the performance of the MPI program and understand the load imbalance
- The profiling tool available on Isca is the “Intel Trace Analyser and Collector” (ITAC) (<https://software.intel.com/en-us/intel-trace-analyzer>)

Workshop 8

Rebuild the executable to enable profiling:

```
> module load intel/2017b
> module load itac/2017.1.024
> mpicc -trace -o mandelbrot mandelbrot_mpi.c -lm
```

Load extra module

Flag to enable collection of profiling information

Workshop 8

Submit the job to the queue:

```
#!/bin/bash
# Number of nodes required
#SBATCH --nodes=1
# Number of tasks per node
#SBATCH --ntasks-per-node=4 ← 4 MPI processes
# Number of CPUs per task
#SBATCH --cpus-per-task=1
# Maximum time the job will run for (HH:MM:SS)
#SBATCH --time=5:00
# Name of the job
#SBATCH --job-name=Mandelbrot_v1_prof
# Queue for the job to run in: pq is the parallel queue
#SBATCH -p pq
# Specify the account code and reservation
#SBATCH -A undergraduate_teaching-ecmm461
#SBATCH --reservation=undergraduate_teaching_2021

# Unload all existing modules
module purge
# Load the Intel Cluster Toolkit module used to compile the program
module load intel/2017b
# Load the Intel Trace analyzer and Collector module
module load itac/2017.1.024 ← Load the ITAC module

mpirun ./mandelbrot ← Run as normal

# End of file
```

Workshop 8

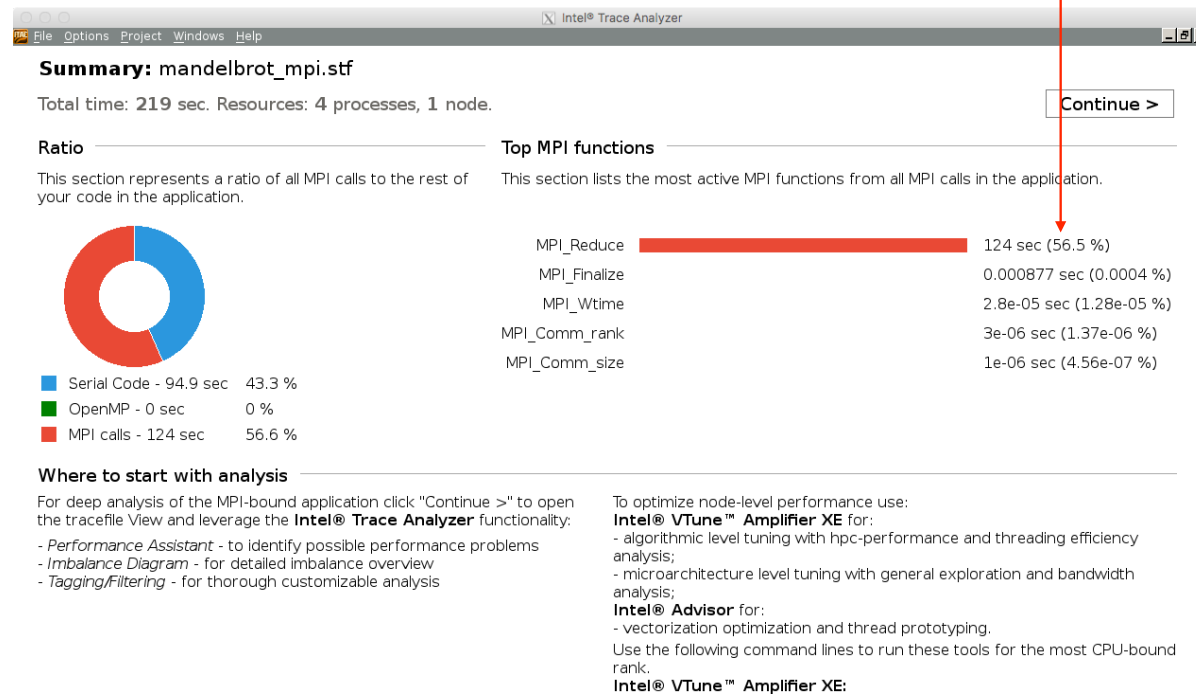
Extra output files will be produced including a trace file called `mandelbrot.stf`

```
> ls
mandelbrot          mandelbrot.stf.cop.anc  mandelbrot.stf.f.1    mandelbrot.stf.f.3    run_mb_prof.sh
mandelbrot.prot     mandelbrot.stf.dcl     mandelbrot.stf.f.1.anc mandelbrot.stf.f.3.anc slurm-510712.out
mandelbrot.stf      mandelbrot.stf.f.0     mandelbrot.stf.f.2    mandelbrot.stf.sts
mandelbrot.stf.cop  mandelbrot.stf.f.0.anc mandelbrot.stf.f.2.anc mandelbrot_mpi.c
```

Open the trace file in the traceanalyzer GUI by running:
`traceanalyzer.bin mandelbrot_mpi.stf &`

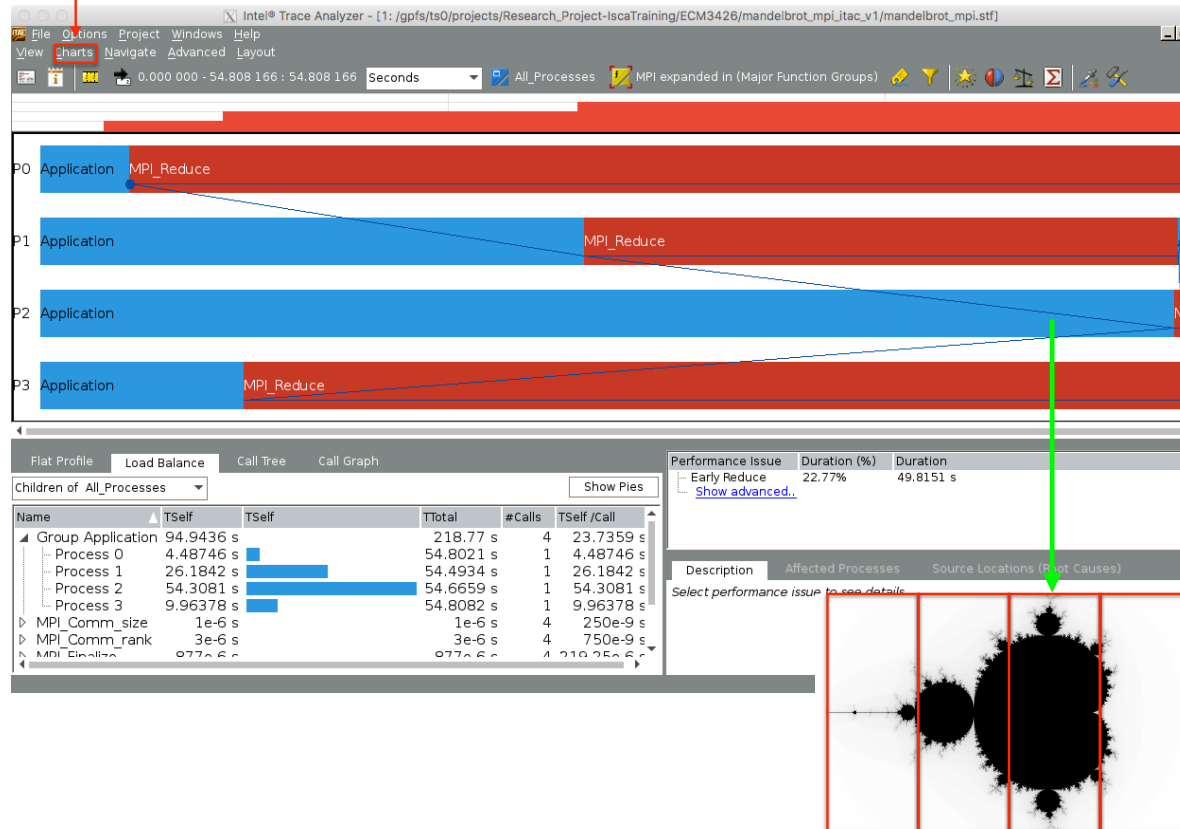
Workshop 8

More than half the time is spent in **MPI_Reduce**



Workshop 8

Charts → Event Timeline



Profiling

- Measuring load imbalance is just one use for a profiler
- Profilers can also measure time spent in functions/subroutines, record hardware performance counters etc
- Use a profiler to base your tuning decisions on evidence
- Different profilers are available: need to find out what is available on any given HPC system

Improving the load balance

- Processes spend a long time waiting at the `MPI_REDUCE` for the final process to catch up
- Work can be distributed differently to give better load balance
- Interleaving loop iterations better balances the load as work in the computationally expensive regions is more evenly distributed (version 2)

Workshop 8

Question 1: Which version of the program performs best and how does it distribute the workload?

Version 2 interleaves iterations of the `i` loop:

```
printf("Process %d of %d starting calculation\n", myRank, nProcs);

/* Initialise to nIter zero as we will use a reduce to collate results into this array*/
for (i=0; i<nRe+1; i++){
    for (j=0; j<nIm+1; j++){
        nIter[i][j]=0;
    }
}

/* Loop over real and imaginary axes interleaving iterations between MPI processes */
for (i=myRank; i<nRe+1; i+=nProcs){
    for (j=0; j<nIm+1; j++){
        z_Re = ( (float) i) / ( (float) nRe) * (z_Re_max - z_Re_min) + z_Re_min;
        z_Im = ( (float) j) / ( (float) nIm) * (z_Im_max - z_Im_min) + z_Im_min;
        z0 = z_Re + z_Im*I;
        z = z0;

        /* Iterate up to a maximum number or bail out if mod(z) > 2 */
        k=0;
        while(k<maxIter){
            nIter[i][j] = k;
            if (cabs(z) > 2.0)
                break;
            z = z*z + z0;
            k++;
        }
    }
}
```

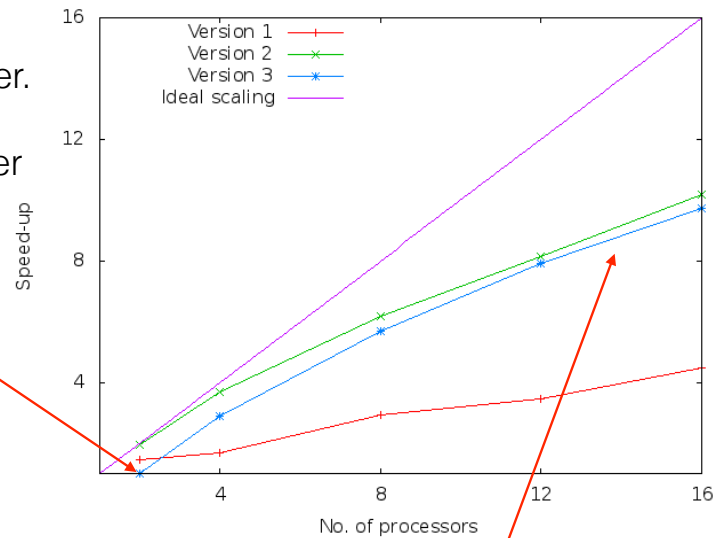
Each process strides through iterations in steps of `nProcs`

Proc 0: 0, 4, 8, 12, ...
Proc 1: 1, 5, 9, 13, ...
Proc 2: 2, 6, 10, 14, ...
Proc 3: 3, 7, 11, 15, ...

Workshop 8

Question 3: Why is version 3 the slowest version with 2 processors?

Version 3 is manager-worker. With 2 processors there is one manager and one worker so only one processor calculates values. This will be no faster than the serial version!

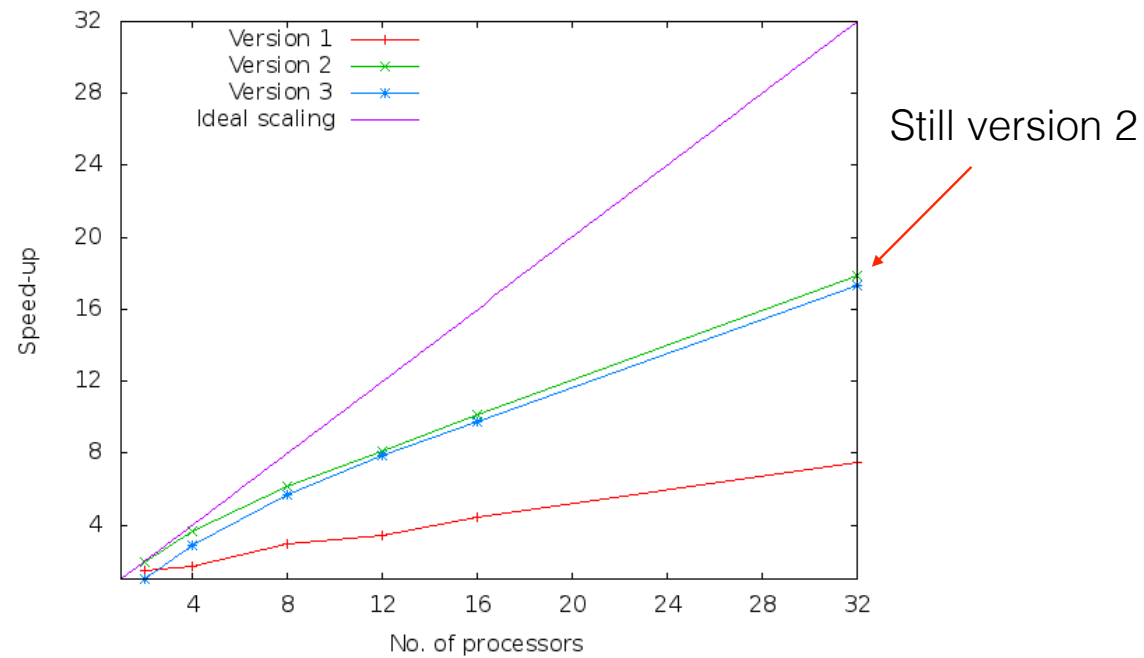


Nearly as fast as version 2 when more processors are used

Workshop 8

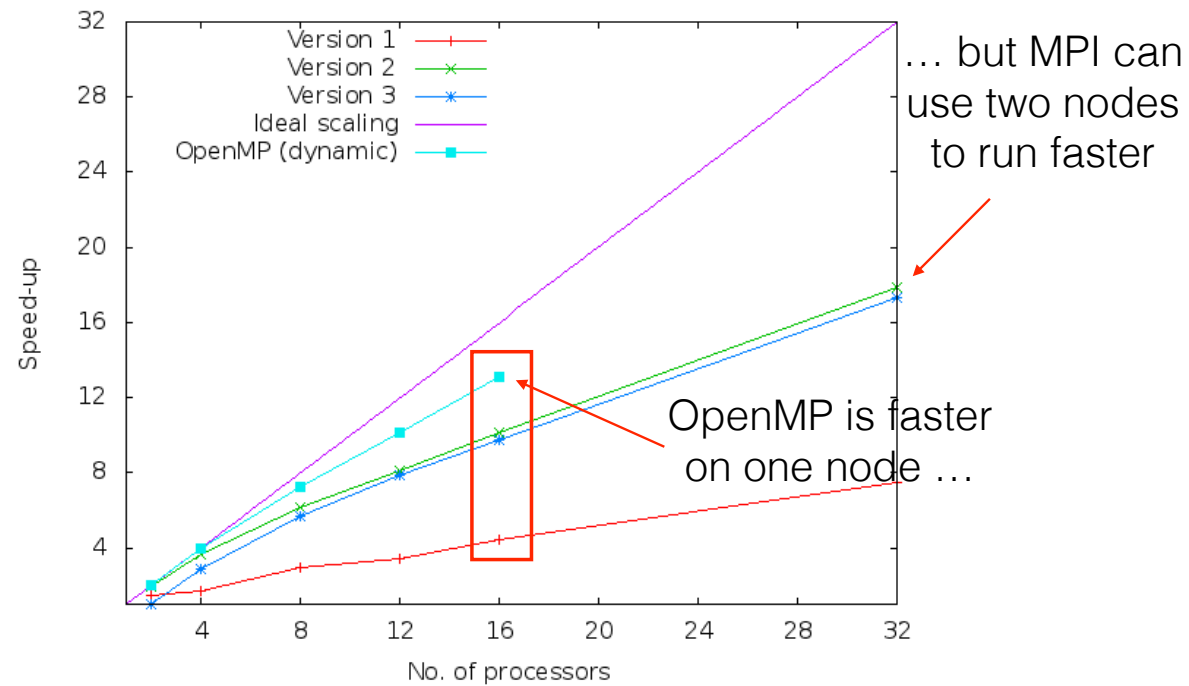
2 node scaling results:

Question 4: Which version of the program performs best running on 2 nodes?

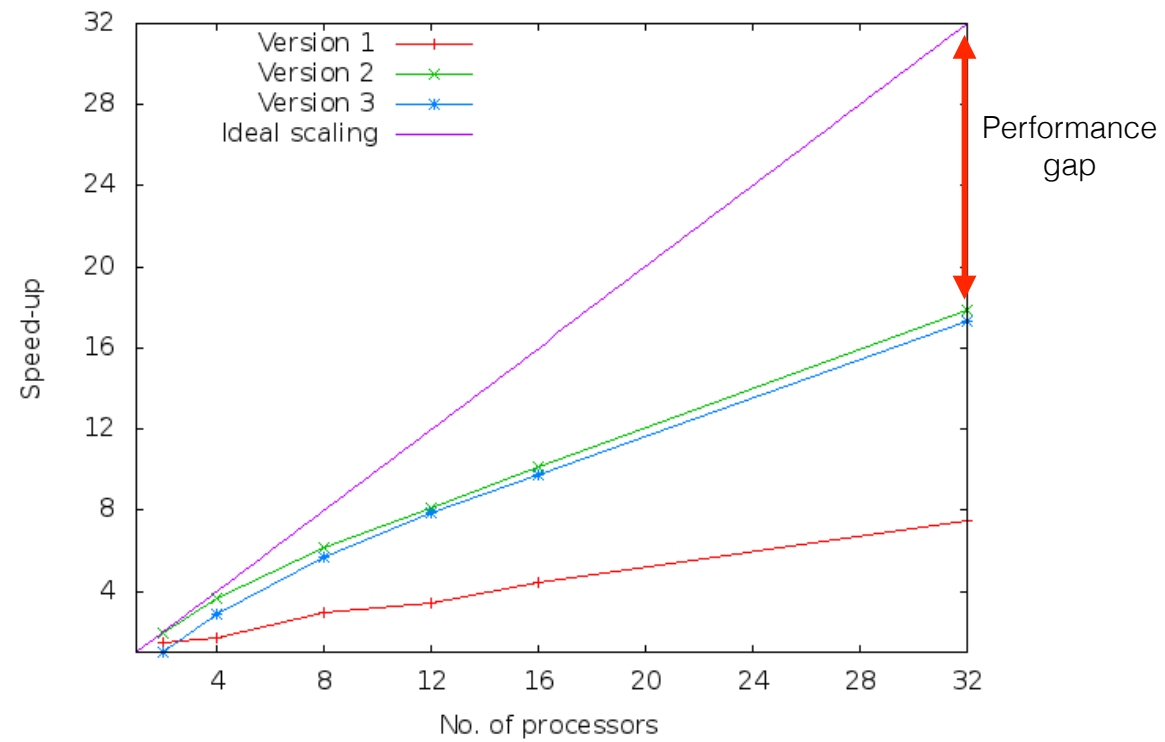


Workshop 8

Question 5: How does the performance of the fastest MPI version compare with the fastest OpenMP version of program?

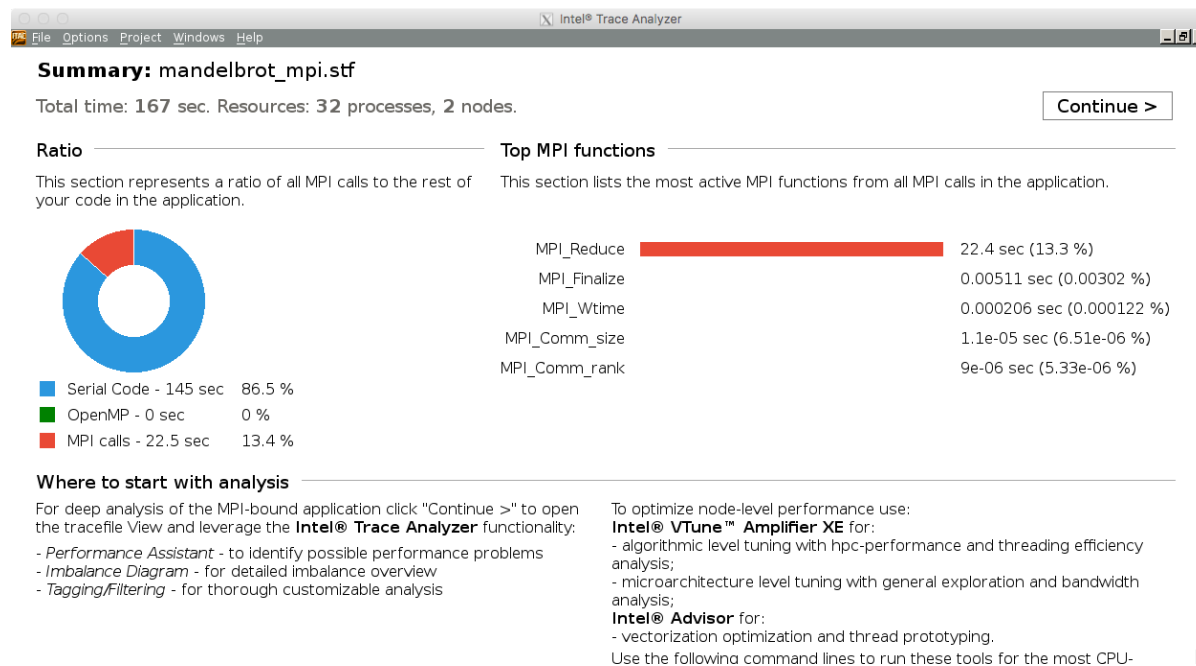


Workshop 8



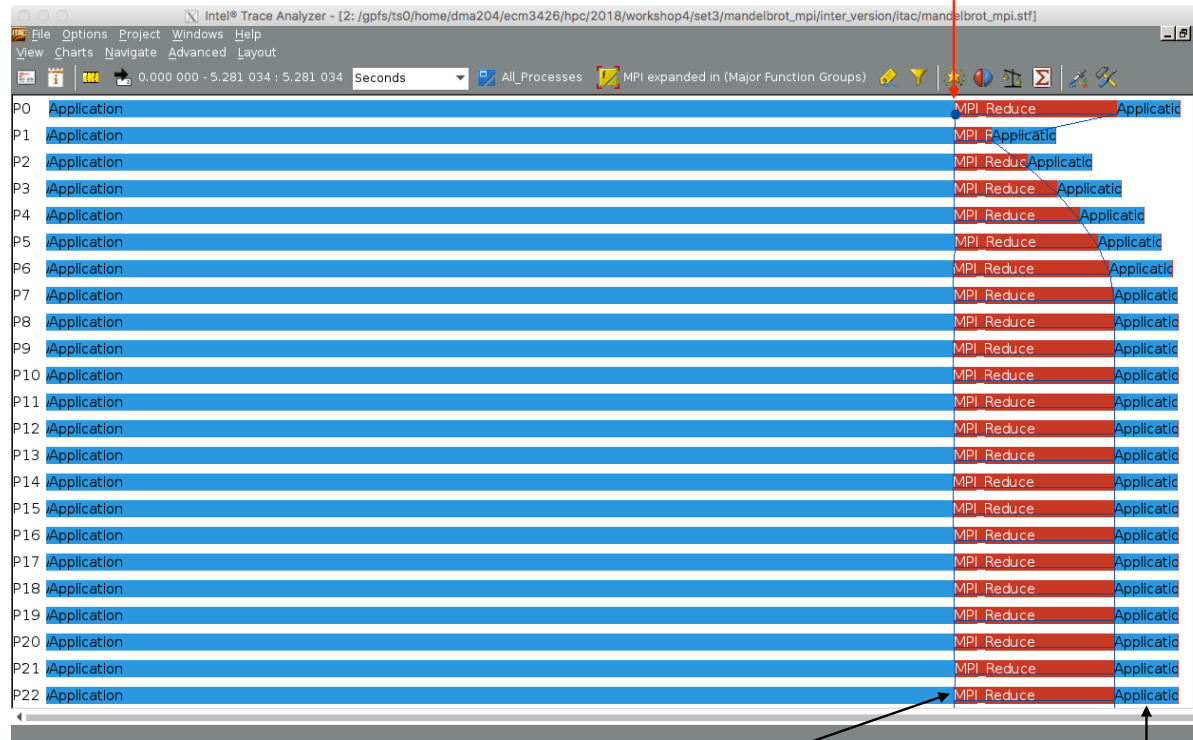
Workshop 8

Version 2 of program using 32 MPI processes on 2 nodes



Workshop 8

Load balancing is OK



MPI_REDUCE Unpack buffer

MPI Mandelbrot version 2

- With the updated method for handing out work (striding by `nProc`) the load balancing problem is fixed
- Scaling is now limited by parallelisation overheads; `MPI_REDUCE` in a serial program we would not pack/unpack buffer or call
- Sending the whole domain with `MPI_REDUCE` is a straightforward way to do the communication but it is not efficient

Workshop 8 summary

- In the Mandelbrot calculation dividing the computational domain into equal chunks is inefficient due to poor load balancing (version 1)
- Interleaving the iterations balances the load better and improves performance (version 2)
- For this program the OpenMP version was fastest on one node but the MPI version 2 could calculate a solution faster using two nodes
- The **MPI_Reduce** takes a significant time to complete so this is a target for optimisation